



**UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA**

PROGETTO: PIATTAFORMA “COWORKSPACE” PER LA GESTIONE DI SPAZI CONDIVISI

Documentazione Tecnica

Laurea Triennale in Informatica

Progetto Tecnologie Innovative per lo Sviluppo Web: Piattaforma “CoWorkSpace” per la gestione di spazi condivisi

Sviluppato da: Mattia Statti, Nicola Pichierri, Manuel Gjolaj, Simone Marino

Sommario

1. Architettura del Sistema

2. Schema ER del Database

3. API del Sistema

- **Struttura Generale**
- **Endpoint Principali**

4. Deploy e Configurazione

- **Requisiti**
- **Avvio Locale**

5. Gestione degli Errori e Rollback

- **Middleware di Error Handling**
- **Strategie di Rollback**

6. Integrazione Servizi Esterni

- **Stripe (Pagamenti)**
- **Chart.js (Frontend)**
- **Leaflet (Mappe)**

7. Sicurezza

8. Possibili Estensioni

1. Architettura del Sistema

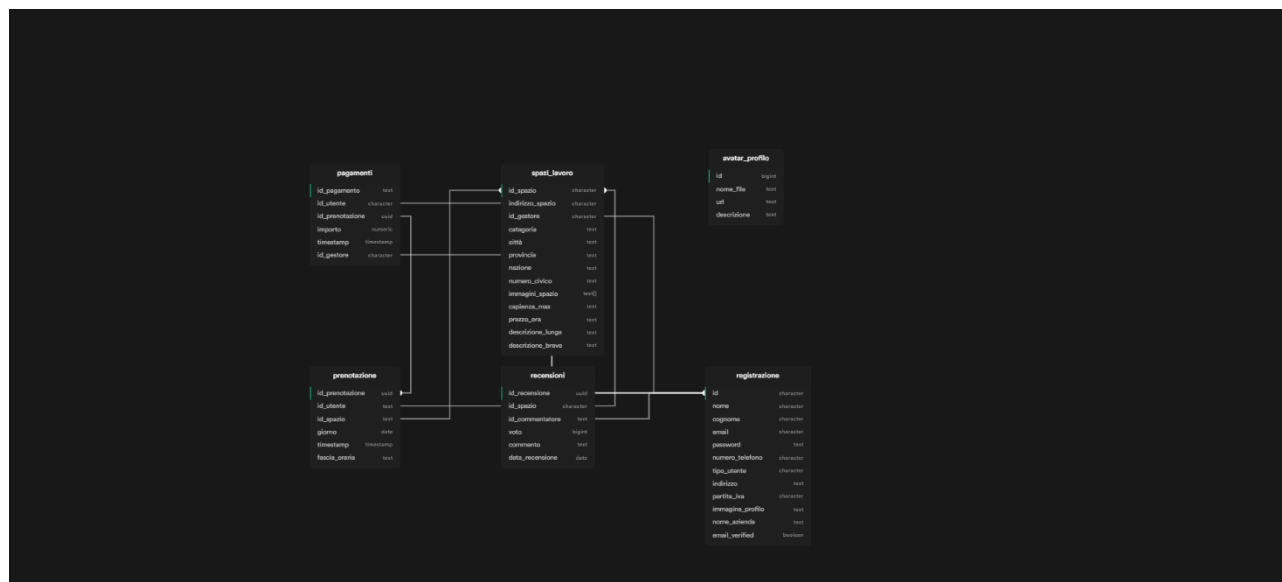
Il sistema è composto da due macro-componenti:

- **Frontend:** sviluppato in HTML, CSS e JavaScript, con l'uso di Chart.js per grafici, Leaflet per mappe interattive e integrazione con il backend per i pagamenti e la gestione dati.
- **Backend:** implementato con Express.js (Node.js), organizzato in un'architettura MVC:
 - **server.js** → avvio del server
 - **app.js** → configurazione middleware e routing
 - **routes/** → definizione endpoint REST
 - **controllers/** → logica applicativa
 - **middlewares/** → gestione errori
 - **config/** → configurazioni (server, Supabase, geocoding)
 - **database/** → funzioni di accesso al database (Supabase)

Tecnologie Esterne

- **Supabase:** per la gestione del database relazionale
- **Stripe:** per la gestione dei pagamenti online

2. Schema ER del Database



Lo schema del database è stato generato con Supabase Schema ed è strutturato come segue:

- **registrazione:** gestisce i dati utente (id, nome, cognome, email, password, tipo utente, azienda, partita IVA, ecc.)



- **spazi_lavoro:** definisce gli spazi disponibili con attributi come indirizzo, gestore, categoria, capacità, prezzo, descrizioni.
- **prenotazione:** collega utenti e spazi con giorno, fascia oraria e timestamp.
- **pagamenti:** registra le transazioni con utente, gestore, importo, prenotazione e timestamp.
- **recensioni:** permette agli utenti di lasciare feedback sugli spazi (commento, voto, data).
- **avatar_profilo:** tabella di supporto per la gestione delle immagini profilo.

Le relazioni principali:

- Un utente può avere più prenotazioni e più pagamenti.
- Ogni prenotazione è collegata a uno spazio_lavoro.
- Ogni spazio_lavoro può avere più recensioni e più prenotazioni.
- I pagamenti collegano utenti, prenotazioni e gestori.

3. API del Sistema

3.1 Struttura Generale

Tutte le API sono esposte sotto il path /api. Le risposte seguono lo schema JSON:

```
{
  "success": true|false,
  "data": {},
  "error": {
    "message": "...",
    "code": 400|500
  }
}
```

3.2 Endpoint Principali

Home Routes (/api/home)

- **GET /api/home** → Recupera i dati iniziali per la homepage.
- **GET /api/home/:id** → Recupera informazioni dettagliate di uno spazio.

Admin Routes (/api/admin)

- **POST /api/admin/spazio** → Creazione nuovo spazio.
- **PUT /api/admin/spazio/:id** → Modifica dati di uno spazio.



- **DELETE /api/admin/spazio/:id** → Rimozione spazio.

Geocoding (/api/geocode)

- **POST /api/geocode** → Richiede la geocodifica di un indirizzo.

Autenticazione e Registrazione (/api/auth)

- **POST /api/auth/register** → Registra un nuovo utente.
- **POST /api/auth/login** → Effettua login e rilascia token.

Pagamenti (Stripe)

- **POST /api/payments/create-session** → Crea una sessione di pagamento Stripe.
- **POST /api/payments/webhook** → Riceve eventi da Stripe (es. pagamento riuscito o fallito).

4. Deploy e Configurazione

4.1 Requisiti

- **Node.js >= 18**
- **Docker e Docker Compose** (opzionale ma raccomandato)
- **Variabili d'ambiente:**
 - **SUPABASE_URL, SUPABASE_KEY** → connessione al DB
 - **STRIPE_SECRET_KEY** → chiave segreta Stripe
 - **STRIPE_PUBLIC_KEY** → chiave pubblica per frontend
 - **PORT** → porta server Express

4.2 Avvio Locale

npm install

npm run dev

Il backend sarà disponibile su <http://localhost:3000>.

5. Gestione degli Errori e Rollback

5.1 Middleware di Error Handling

Il file `middlewares/errorHandler.js` centralizza la gestione degli errori:

- **Restituisce risposte JSON con `success: false`**
- **Logga gli errori lato server**
- **Supporta errori 400 (Bad Request), 401 (Unauthorized), 404 (Not Found), 500 (Server Error)**

5.2 Strategie di Rollback

- **Database (Supabase):** in caso di errore nelle transazioni, gli inserimenti/modifiche vengono annullati.
 - **Stripe:** in caso di errore nel pagamento, la prenotazione non viene confermata e l'utente riceve notifica.
 - **Geocoding:** se la geocodifica fallisce, viene restituito un errore e il flusso si interrompe senza modificare dati persistenti.
-

6. Integrazione Servizi Esterni

6.1 Stripe (Pagamenti)

- Usato per gestire transazioni sicure
- Creazione sessione di pagamento → reindirizzamento utente a checkout Stripe
- Webhook per gestire esito pagamento

6.2 Chart.js (Frontend)

- Utilizzato per grafici statistici nella dashboard gestore
- Dati recuperati via API (/api/admin/spazio) e rappresentati con grafici dinamici

6.3 Leaflet (Mappe)

- Utilizzato per mostrare spazi geolocalizzati
 - Collegato al servizio di geocoding (backend) per convertire indirizzi in coordinate
-

7. Sicurezza

- Uso di HTTPS (consigliato in produzione)
 - Token JWT per autenticazione utenti
 - Validazione input lato backend per prevenire SQL injection e XSS
 - Stripe gestisce direttamente i dati sensibili di pagamento (PCI compliant)
-

8. Possibili Estensioni

- Gestione ruoli e permessi granulari (admin, utente, gestore)
- Logging avanzato con strumenti come Winston o ELK Stack
- Monitoraggio e metriche (Prometheus + Grafana)